# HouseCallPro Integration

## 1. Overview

### 1.1 Description

**HouseCallPro Integration** connects **HouseCallPro** (field service management platform) with **Newo Platform**.

It enables:

- Checking available appointment slots with intelligent employee selection (by role, job type, or zip code)
- Booking appointments with automatic customer search, creation, address comparison, and job management
- Cancelling existing appointments
- Recognizing returning customers on inbound calls with upcoming job context
- Caching company data (employees, job types, locations) with configurable refresh intervals
- Automatic line item matching using AI for job cost estimation

This integration is designed for **home service businesses** (HVAC, plumbing, electrical, cleaning, etc.) who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

### 1.2 Use Cases

- When a customer asks about available slots -> AI selects the right employee and returns open time slots
- When a customer wants to book a service -> Search or create customer record, create job with line items, and schedule appointment
- When a customer wants to cancel -> Cancel the appointment via HouseCallPro API
- When an inbound call arrives -> Automatically look up customer by phone and display upcoming jobs to the agent
- When a customer describes a service need -> AI matches the description to job types and line items for accurate quoting
- When a session starts -> Static data (employees, job types, company info) is refreshed if cache is stale

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Check Availability | Yes | With 3 employee selection methods (POC, job type, zip) |
| Book Appointment | Yes | Creates job + appointment with customer management |
| Cancel Appointment | Yes | Requires booking_id and job_id from prior booking |
| Employee Selection (POC) | Yes | Assigns Point of Contact employee (default method) |
| Employee Selection (Job Type) | Yes | AI matches job description to employee skills/tags |

| | | |
|---|---|---|
| Employee Selection (Zip Code) | Yes | Matches employee by service area zip code tags |
| Customer Search | Yes | By phone number via HouseCallPro API |
| Customer Creation | Yes | Auto-creates during booking flow with address |
| Address Comparison | Yes | Compares provided address with HCP profile, adds new address if different |
| Existing Client Recognition | Yes | Looks up customer + upcoming jobs on inbound call |
| Auto Line Item Matching | Yes | AI matches service description to catalog items |
| Static Data Caching | Yes | Employees, job types, company info with 24h TTL |
| Multi-Location Support | Yes | Location-specific employee filtering |
| Token Auto-Refresh | No | Uses static API key (Bearer token) |
| Reschedule Appointment | No | Cancel + rebook as workaround |
| Historical Data Import | No | Only new events after activation |

## 2. Requirements

Before installation:

- Active **HouseCallPro** account (Pro plan or higher for API access)
- **HouseCallPro API key** for authentication
- At least one employee configured with appropriate permissions
- For POC method: at least one employee marked as Point of Contact in HouseCallPro

## 3. How to Setup

### 3.1 Step 1 — Get API Credentials

1. Log in to your **HouseCallPro** account
2. Navigate to account settings or developer/API section
3. Generate or locate your **API key**
4. Note your company timezone setting

### 3.2 Step 2 — Connect in Platform

1. Open **Newo -> Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `hcp_api_key` | Yes | HouseCallPro API key for authentication |
| `hcp_base_url` | No | API base URL (default: `https://api.housecallpro.com/`) |
| `hcp_enable_slot_check` | No | Enable availability checks (default: `True`) |

| | | |
|---|---|---|
| `hcp_enable_booking` | No | Enable appointment creation (default: `True`) |
| `hcp_enable_cancellation` | No | Enable cancellations (default: `True`) |
| `hcp_auto_employee_type` | No | Employee assignment method: `poc`, `job_type`, or `zip_code` (default: `poc`) |
| `hcp_booking_duration` | No | Appointment duration in hours (default: `2`) |
| `hcp_show_for_days` | No | Days to search for availability (default: `5`) |
| `hcp_availability_offset` | No | Days before first available slot (default: `2`) |
| `hcp_check_existing_client` | No | Look up customer on inbound call (default: `True`) |
| `hcp_auto_add_line_items` | No | Auto-match line items to jobs via AI (default: `False`) |
| `hcp_line_items` | No | Service catalog as JSON array of items |
| `hcp_job_type_employee_selection_prompt` | No | Custom LLM prompt for job-type employee matching |
| `hcp_static_data_update_interval` | No | Cache refresh interval in seconds (default: `86400`) |

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Creates an HTTP connector for all API operations
    - Fetches and caches company info, employees, and job types
    - Registers availability, booking, and cancellation tools with the agent
    - Injects payload schemas for the ConvoAgent tool calling framework

## 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

### 4.1 How the Integration Works

The integration uses HouseCallPro's REST API for job management, appointment scheduling, customer records, and employee data. A key architectural feature is the two-level booking model: a **Job** is created first (representing the service request), then an **Appointment** is attached to it (representing the scheduled time slot). Employee selection supports three AI-powered strategies. Static data is cached with a configurable TTL to minimize API calls.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in HouseCallPro

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Check Availability** | `get_available_slots` | When the agent needs to look up open slots |

| | | |
|---|---|---|
| **Book Appointment** | `book_slot` | When the agent confirms a booking |
| **Cancel Appointment** | `convo_agent_cancel_booking` | When the agent processes a cancellation |
| **Session Start** | `session_started` | Refreshes static data cache if stale |
| **Inbound Call** | `define_user_phone_number` | Looks up existing customer and upcoming jobs |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |
| **Session End** | `end_session` | Clears temporary customer data |

**Available Actions**

- **Get Company** — Fetch company info and locations (GET `/company`)
- **Get Job Types** — Fetch available service types (GET `/job_fields/job_types`)
- **Get Employees** — Fetch employees, optionally by location (GET `/employees`)
- **Get Booking Windows** — Fetch available time slots (GET `/company/schedule_availability/booking_windows`)
- **Get Jobs** — Fetch scheduled jobs for conflict detection (GET `/jobs`)
- **Get Events** — Fetch calendar events for conflict detection (GET `/events`)
- **Search Customer** — Find existing customer by phone (GET `/customers`)
- **Create Customer** — Create a new customer record (POST `/customers`)
- **Add Customer Address** — Add a new address to an existing customer (POST `/customers/{id}/addresses`)
- **Create Job** — Create a service job (POST `/jobs`)
- **Create Appointment** — Schedule a time slot on a job (POST `/jobs/{id}/appointments`)
- **Cancel Appointment** — Delete a scheduled appointment (DELETE `/jobs/{id}/appointments/{apt_id}`)
- **Get Job Appointments** — Fetch appointments for a job (GET `/jobs/{id}/appointments`)

## 4.2 Architecture Diagrams

**Overall Sequence**

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as HCP Integration
    participant API as HouseCallPro API

    Note over I,API: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: GET /company
    I->>API: GET /job_fields/job_types
    I->>API: GET /employees
    I->>A: Tools registered, data cached

    Note over I,API: Inbound Call
    A->>I: define_user_phone_number
    I->>API: GET /customers?q={phone}
```

```
    API-->>I: Customer data
    I->>API: GET /jobs?customer_id={id}
    API-->>I: Upcoming jobs
    I->>A: ExistingCustomerInfo injected

    Note over U,API: Availability Check
    U->>A: "Any slots this week?"
    A->>I: get_available_slots (job_description, zip_code)
    I->>I: Select employees (POC/job_type/zip)
    I->>API: GET /schedule_availability/booking_windows
    API-->>I: Available windows
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book Tuesday 10 AM"
    A->>I: book_slot (customerInfo, address, dateTime)
    I->>API: GET /customers?q={phone}
    alt Customer not found
        I->>API: POST /customers
    else Customer found, address differs
        I->>API: POST /customers/{id}/addresses
        Note over I: "Address updated" added to notes
    end
    I->>API: POST /jobs (customer_id, address_id, notes)
    I->>API: POST /jobs/{id}/appointments (time, employee)
    API-->>I: appointment_id
    I->>A: result_success (booking_id, job_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id, job_id)
    I->>API: DELETE /jobs/{job_id}/appointments/{booking_id}
    API-->>I: Confirmation
    I->>A: result_success (cancelled)
    A->>U: "Your appointment has been cancelled"
```

**StaticDataFlow (Caching)**

```
flowchart TD
    E["session_started /<br>hcp_update_static_data"] --> CHK["Check cache age:
<br>updated_at vs now"]
    CHK --> STALE{"Cache older than<br>update_interval?"}
    STALE -->|"No"| SKIP["Use cached data"]
    STALE -->|"Yes"| COMP["GET /company<br>Extract locations, timezone"]
    COMP --> TYPES["GET /job_fields/job_types<br>Cache all job types"]
    TYPES --> LOC{"Multiple<br>locations?"}
    LOC -->|"No"| EMP["GET /employees?page_size=100"]
    LOC -->|"Yes"| EMP_LOC["GET /employees<br>per location_id"]
```

```
    EMP --> SAVE["Save to hcp_company_employees<br>Update timestamp"]
    EMP_LOC --> SAVE_LOC["Save to hcp_location_employees<br>Update timestamp"]
```

**AvailabilityFlow (Employee Selection)**

```
flowchart TD
    E["get_available_slots"] --> CHK{"Slot check<br>enabled?"}
    CHK -->|"No"| SKIP["Return (disabled)"]
    CHK -->|"Yes"| METHOD{"Employee<br>selection method?"}

    METHOD -->|"poc"| POC["Filter employees:<br>is_point_of_contact = true"]
    METHOD -->|"job_type"| JT["LLM matches job_description<br>to employee
tags/skills<br>(Gemini 2.5 Flash, temp 0.15)"]
    METHOD -->|"zip_code"| ZIP["Filter employees:<br>tags contain customer zip"]

    POC --> POC_API["GET /schedule_availability<br>/booking_windows<br>?
show_for_days=...&start_date=..."]
    POC_API --> POC_PARSE["Parse booking windows:<br>filter available=true,<br>group
by date"]

    JT --> FALLBACK{"Employees<br>found?"}
    ZIP --> FALLBACK
    FALLBACK -->|"No"| POC
    FALLBACK -->|"Yes"| MULTI["For each employee:"]
    MULTI --> JOBS["GET /jobs<br>?employee_ids[]={id}<br>&scheduled_start_min=
{date}"]
    JOBS --> EVENTS["GET /events<br>?page_size=20"]
    EVENTS --> CONFLICT["Detect conflicts:<br>exclude busy intervals<br>from
available windows"]
    CONFLICT --> BEST["Select employee with<br>earliest available slot"]

    POC_PARSE --> OUT["result_success<br>{availability[]}"]
    BEST --> OUT
```

**BookingFlow**

```
flowchart TD
    E["book_slot"] --> CHK{"Booking<br>enabled?"}
    CHK -->|"No"| SKIP["Return (disabled)"]
    CHK -->|"Yes"| PHONE["Format phone:<br>strip +, remove leading 1"]
    PHONE --> SEARCH["GET /customers?q={phone}"]
    SEARCH --> FOUND{"Customer<br>found?"}
    FOUND -->|"Yes"| ADDR{"Address matches<br>HCP profile?<br>(street + zip)"}
    FOUND -->|"No"| CREATE["POST /customers<br>{first_name, last_name,<br>phone,
email, address}"]
    CREATE --> NEW["Extract new<br>customer_id + address_id"]
    ADDR -->|"Yes"| USE["Use existing<br>address_id"]
    ADDR -->|"No"| ADD["POST /customers/{id}/addresses<br>Add new address"]
    ADD --> NOTE["Append 'Address updated'<br>to job notes"]
    NOTE --> JOB["POST /jobs<br>{customer_id, address_id, notes}"]
```

```
    USE --> JOB
    NEW --> JOB
    JOB --> LINE{"Auto line items<br>enabled?"}
    LINE -->|"Yes"| LLM["LLM matches description<br>to line_items catalog<br>and
job_types"]
    LINE -->|"No"| EMP["Get selected employee<br>(from availability step)"]
    LLM --> EMP
    EMP --> APPT["POST /jobs/{job_id}/appointments<br>{start_time (UTC),<br>end_time
(+duration hours),<br>dispatched_employees_ids[]}"]
    APPT --> OK{"Success?"}
    OK -->|"No"| ERR["result_error"]
    OK -->|"Yes"| OUT["result_success<br>{booking_id, job_id}"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> CHK{"Cancellation<br>enabled?"}
    CHK -->|"No"| SKIP["Return (disabled)"]
    CHK -->|"Yes"| VAL{"booking_id and<br>job_id present?"}
    VAL -->|"No"| ERR["result_error:<br>missing IDs"]
    VAL -->|"Yes"| API["DELETE /jobs/{job_id}<br>/appointments/{booking_id}"]
    API --> OK{"Success?"}
    OK -->|"No"| ERR2["result_error"]
    OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]
```

**ExistingClientFlow (Caller Recognition)**

```
flowchart TD
    E["define_user_phone_number"] --> CHK{"Check existing<br>client enabled?"}
    CHK -->|"No"| SKIP["Skip lookup"]
    CHK -->|"Yes"| SEARCH["GET /customers?q={phone}"]
    SEARCH --> FOUND{"Customer<br>found?"}
    FOUND -->|"No"| DONE["No context injected"]
    FOUND -->|"Yes"| INFO["Extract: name, email,<br>phone, address"]
    INFO --> JOBS["GET /jobs?customer_id={id}<br>&scheduled_start_min={today}"]
    JOBS --> APPTS["For each job:<br>GET /jobs/{id}/appointments"]
    APPTS --> INJECT["Inject ExistingCustomerInfo<br>into agent context"]
    INJECT --> DONE2["Agent sees customer<br>history on greeting"]
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (voice or chat). Each flow has a dedicated webhook test endpoint:

- `availability_test_event` — AvailabilityFlow
- `appointment_test_event` — AppointmentFlow
- `appointment_test_cancel_event` — CancellationFlow
- `get_client_test_event` — ExistingClientFlow
- `update_static_test_event` — StaticDataFlow

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish the project and verify static data is cached (check `hcp_company_employees` is populated)
2. **Static Data**: Verify `hcp_company_info`, `hcp_job_types`, and `hcp_time_zone` are populated
3. **Existing Client**: Call from a known customer's phone — verify agent greets with context
4. **Availability**: Ask "What slots are available this week?" — verify slots returned with correct times
5. **Booking**: Complete a booking conversation with address — verify job and appointment appear in HouseCallPro
6. **Cancellation**: Request cancellation — verify appointment removed from HouseCallPro

**If no action occurs:**

- Ensure `hcp_api_key` is set correctly and has appropriate permissions
- Verify static data was cached (check `hcp_company_employees` is not empty)
- Confirm the relevant feature flags are enabled ( `hcp_enable_booking`, `hcp_enable_slot_check`, `hcp_enable_cancellation` )
- For POC method: verify at least one employee has Point of Contact permission enabled in HouseCallPro
- For job_type method: verify employees have relevant tags/skills configured
- For zip_code method: verify employees have zip code tags matching the service area
- Check that `hcp_booking_duration` and `hcp_show_for_days` are reasonable values
- Re-publish the project if the API key was changed

> *Note: This integration performs real operations in HouseCallPro. Jobs, appointments, and customers created or cancelled through the agent are reflected in the live HouseCallPro system.*

## 5. FAQ

**Q: What are the three employee selection methods and when should I use each?** A: **POC** (default) assigns the Point of Contact employee — best for single-technician businesses or when one person handles all dispatch. **Job Type** uses AI to match the customer's service description to employee skills/tags — best for multi-specialty teams. **Zip Code** matches by service area — best for geographically distributed teams. Set via `hcp_auto_employee_type`.

**Q: How does the AI line item matching work?** A: When `hcp_auto_add_line_items` is enabled, the integration uses Gemini 2.5 Flash to match the customer's service description against the `hcp_line_items` catalog and `hcp_job_types`. The AI selects the most relevant items and attaches them to the job for accurate quoting.

**Q: What customer information is required for booking?** A: First name, phone number (E.164 format), and full service address (street, city, state, zip, country) are required. Last name defaults to the first name if not provided. Email is optional.

**Q: How often is static data refreshed?** A: By default every 24 hours ( `hcp_static_data_update_interval` = 86400 seconds). The cache is checked on each session start. You can trigger a manual refresh via the `hcp_update_static_data` event or by re-publishing the project.

**Q: What is the default appointment duration?** A: 2 hours, configurable via `hcp_booking_duration`. The end time is calculated as start time + duration in hours. Times are converted from the business timezone to UTC for the API.

**Q: How does existing customer recognition work?** A: When `hcp_check_existing_client` is enabled, the integration looks up the caller's phone number on inbound calls. If a customer is found, their profile and upcoming jobs are injected into the agent's context as `ExistingCustomerInfo`, allowing the agent to greet them by name and reference their scheduled services.

**Q: What happens if the customer provides a different address than what's in their HCP profile?** A: The integration compares the provided address against all stored addresses by **street + zip code** (case-insensitive). If no match is found, the new address is automatically added to the customer's profile via `POST /customers/{id}/addresses`, and the job notes include `"Address updated: {new address}"` so dispatchers are aware. The job is created with the new address ID.

**Q: What happens if no employees match the selected method?** A: The integration falls back to the POC method. If no POC employee is found either, the availability check will return no slots and the agent will inform the customer accordingly.